

FMC 2026

Controles intuitivos e redução da carga mental do piloto

Sumário

1. O sistema de controle deve ser pensado desde o primeiro dia	3
Os 6 princípios apresentados.....	3
2. Um único piloto	3
3. Retirar o operador pode liberar uma pessoa para estratégia	4
4. Se forem necessários dois operadores, evitar sincronização	5
5. Uma provocação importante da palestra	5
6. O software deve ser o copiloto	6
O humano é melhor em:	6
O software é melhor em:	6
7. Botões devem representar intenções, não motores ou mecanismos.....	6
8. Exemplo aplicado ao Intake.....	7
9. Feedback tátil para o piloto	8
10. O software pode tentar interpretar a intenção do piloto	9
11. A transição humano → automação deve ser praticamente invisível	9
12. Evitar Toggle	10
13. Automação pouco confiável pode ser pior que nenhuma automação ...	10
14. Controles manuais de emergência têm valor limitado	11
15. Segurança deve estar incorporada ao código	11
16. Proteção contra colisões com o campo.....	12
17. IMU também pode ser utilizada como dispositivo de segurança.....	12
18. O software também deve impedir penalidades	12

Ergonomia do controle	13
19. Os melhores botões são aqueles que não exigem remover os dedos dos analógicos.....	13
20. Evitar clicar nos analógicos	13
21. D-Pad para funções menos frequentes	14
22. Desgaste físico dos controles importa.....	14
23. Resposta do joystick deve favorecer memória muscular.....	15
24. Logar também os controles do piloto	15
25. Treinar desde cedo com o layout definitivo	16
26. Mudar cedo e travar cedo.....	16
Conclusão.....	17
Principal aprendizado.....	17

1. O sistema de controle deve ser pensado desde o primeiro dia

A principal ideia da palestra é que o **driver deve fazer parte das decisões de projeto desde o início**.

Enquanto o time projeta a mecânica, já deveria perguntar:

Como o piloto vai controlar isso durante uma partida?

Se um mecanismo adiciona muitos graus de liberdade e exige diversas ações manuais para funcionar, ele pode estar tornando o robô pior, mesmo que mecanicamente seja interessante.

A filosofia da equipe é bastante forte:

Se uma funcionalidade aumenta muito a carga mental do piloto e não pode ser adequadamente automatizada, pode ser melhor não colocá-la no robô.

Isso muda a maneira de pensar o projeto: **complexidade mecânica precisa ser compatível com a capacidade de software e operação da equipe**.

Os 6 princípios apresentados

A equipe utiliza seis princípios para desenvolver os controles:

1. **Um único driver controlando o máximo possível do robô.**
2. **Software como parceiro do piloto.**
3. **Controles previsíveis e sem estados confusos.**
4. **Segurança incorporada ao software.**
5. **Ergonomia na distribuição dos comandos.**
6. **Refinamento baseado em dados e logs.**

Esses seis pontos praticamente formam uma metodologia para desenvolver a interface de pilotagem.

2. Um único piloto

A preferência da equipe é que **uma única pessoa tenha controle sobre praticamente todas as ações importantes do robô**.

O problema de dividir excessivamente as funções entre driver e operator é a necessidade de sincronização.

Imagine:

Driver: posiciona o robô.

Operator: posiciona mecanismo.

Driver: confirma posição.

Operator: dispara.

Mesmo que essa comunicação leve apenas algumas frações de segundo, ela acontece várias vezes durante a partida.

Com um único piloto:

chegar → pressionar comando → pontuar.

Sem comunicação verbal intermediária.

Isso também facilita muito a criação de **memória muscular**, porque o piloto pode praticar todo o ciclo sozinho.

3. Retirar o operador pode liberar uma pessoa para estratégia

Esse ponto é muito interessante.

Se o robô não precisa de um segundo operador dedicado aos mecanismos, essa pessoa pode assumir outras funções permitidas pelo jogo.

Dependendo da temporada, ela pode:

- atuar como Human Player;
- observar peças disponíveis;
- informar locais de pontuação;
- auxiliar estrategicamente;
- acompanhar tempos e estados importantes da partida.

Em 2026, por exemplo, o palestrante cita a possibilidade de essa pessoa auxiliar no acompanhamento dos tempos relacionados aos períodos de pontuação.

Portanto, automação não serve apenas para tornar o robô melhor: ela pode **liberar capacidade humana para estratégia**.

4. Se forem necessários dois operadores, evitar sincronização

O palestrante reconhece que alguns robôs podem precisar de duas pessoas.

Nesse caso, ele recomenda separar as responsabilidades de forma **assíncrona**.

Por exemplo:

O operador informa:

“A próxima pontuação será alta.”

O software prepara o mecanismo.

Mas quem determina **o momento exato da pontuação continua sendo o driver**.

O que deve ser evitado é algo como:

“Vai um pouco para esquerda... para... agora... pode disparar!”

Esse tipo de sincronização adiciona atraso e possibilidade de erro.

5. Uma provocação importante da palestra

O palestrante resume sua filosofia com uma afirmação propositalmente polêmica:

Se você precisa de dois drivers para operar o robô, talvez seu robô seja complexo demais para o seu software.

Não significa simplesmente abolir o operador.

A mensagem é:

complexidade mecânica + capacidade de software + capacidade do piloto precisam evoluir juntas.

Construir um robô com muitos graus de liberdade sem capacidade de automatizá-los pode gerar um robô teoricamente sofisticado, porém difícil de operar.

6. O software deve ser o copiloto

Essa talvez seja a ideia mais importante de toda a palestra.

O palestrante divide as responsabilidades de acordo com aquilo que humanos e computadores fazem melhor.

O humano é melhor em:

- consciência espacial;
- perceber adversários;
- adaptar trajetórias;
- evitar parceiros;
- estratégia;
- tomada de decisão;
- adaptação ao caos.

O software é melhor em:

- precisão;
- repetibilidade;
- sequenciamento;
- gerenciamento de estados;
- baixa latência;
- execução de tarefas repetitivas.

Portanto, não faz sentido o software tentar substituir o piloto em tudo.

A proposta é:

Humano decide o que quer fazer. Software determina como executar.

7. Botões devem representar intenções, não motores ou mecanismos

Esse conceito é extremamente importante para nossos projetos.

Um sistema tradicional poderia ter:

- **Botão A:** subir elevador
- **Botão B:** posicionar pulso
- **Botão X:** abrir outtake

O piloto precisa mentalmente executar:

A → esperar → B → esperar → X.

A proposta apresentada é completamente diferente.

Ter apenas:

Botão: PONTUAR

O piloto informa sua intenção.

Então o software:

1. verifica o estado do robô;
2. termina ações anteriores;
3. posiciona o elevador;
4. posiciona o mecanismo;
5. alinha;
6. verifica condições;
7. libera a peça.

O piloto não está controlando mecanismos.

Ele está dando ordens de alto nível ao robô.

8. Exemplo aplicado ao Intake

Em vez de:

- **Botão → baixar Intake**
- **Botão → ligar rollers**
- **Botão → recolher intake**

O piloto simplesmente mantém:

COLETAR

O software então:

- posiciona o intake;
- liga os motores;
- detecta a peça;
- informa ao piloto;
- armazena a peça;
- encerra a coleta adequadamente.

Isso também resolve outro problema.

Se o piloto soltar o botão no momento errado, o robô **não deveria abandonar uma peça pela metade.**

O software entende que a solicitação anterior ainda precisa terminar de maneira segura.

9. Feedback tátil para o piloto

A comunicação não deve acontecer somente:

Piloto → Robô

Também deve existir:

Robô → Piloto

Uma das soluções utilizadas é a vibração do controle.

Por exemplo:

Rumble → peça adquirida.

Assim, o piloto não precisa:

- olhar para o intake;
- olhar para LEDs;
- perguntar ao operador;
- verificar dashboard.

Ele simplesmente **sente que o robô coletou a peça.**

A equipe também utilizou vibração para indicar situações de ambiguidade de posicionamento, permitindo que o piloto soubesse que o software poderia interpretar aquela situação de maneiras diferentes.

Esse é um recurso muito interessante para implementarmos.

10. O software pode tentar interpretar a intenção do piloto

Outro detalhe avançado apresentado foi considerar não somente:

onde o robô está

mas também:

para onde ele está indo.

Se o robô está próximo da fronteira entre duas regiões, por exemplo, apenas a posição pode gerar ambiguidade.

Mas:

posição + velocidade + direção

podem indicar qual era a intenção do piloto.

Isso permite que o software antecipe a ação correta.

11. A transição humano → automação deve ser praticamente invisível

O palestrante mostrou um exemplo do robô de 2025.

O piloto estava conduzindo normalmente e, em determinado momento, a automação assumia o alinhamento para pontuação.

A intenção era que fosse difícil identificar exatamente:

“Aqui o humano parou e aqui o automático começou.”

A trajetória deveria continuar suave.

Isso é uma excelente referência para nossas automações:

Driver assist não deveria parecer um modo separado de pilotagem.

Ele deveria parecer simplesmente que **o robô está ajudando o piloto.**

12. Evitar Toggle

A equipe evita comandos do tipo:

- **Aperta uma vez → liga Intake**
- **Aperta novamente → desliga intake**

O problema é que agora existe um estado interno que o piloto precisa lembrar.

“O intake está ligado ou desligado?”

Isso adiciona carga mental.

A preferência é por controles cujo resultado seja previsível.

Ao pegar o controle, o piloto deveria conseguir operar sem precisar primeiro descobrir **em qual modo o robô está**.

13. Automação pouco confiável pode ser pior que nenhuma automação

Outro ponto excelente:

Uma automação que funciona algumas vezes pode ser pior que não possuir automação.

Se funciona 80% das vezes, o piloto precisa desenvolver duas memórias musculares:

- **quando funciona → fazer A**
- **quando falha → fazer B**

Isso aumenta significativamente a carga mental.

A equipe só começa realmente a confiar em automações quando elas estão próximas de **99% de confiabilidade**, segundo o palestrante.

Portanto:

Automação → testar → tornar extremamente confiável → incorporar ao estilo de pilotagem.

14. Controles manuais de emergência têm valor limitado

Essa também foi uma opinião provocativa do palestrante.

É comum criarmos:

modo automático + modo manual completo de emergência.

Porém, se o piloto quase nunca pratica o modo manual, quando a automação quebrar em uma eliminatória provavelmente ele **não terá memória muscular suficiente para utilizá-lo corretamente.**

Por isso, em alguns casos, o tempo gasto desenvolvendo dezenas de overrides poderia ser melhor empregado tornando a automação extremamente confiável.

Ele não defende eliminar absolutamente todo override, mas questiona a utilidade de criar controles manuais para tudo.

15. Segurança deve estar incorporada ao código

O piloto não deveria conseguir destruir o robô apertando uma combinação errada de botões.

Exemplo:

Se:

Braço + Intake = colisão

o software deve impedir essa combinação.

Melhor ainda: se a equipe não consegue garantir isso via software, considerar **projetar mecanicamente o robô de forma que essa colisão seja impossível.**

O palestrante é bastante categórico:

O robô não deve ser capaz de se destruir simplesmente porque o piloto apertou botões.

16. Proteção contra colisões com o campo

O mesmo princípio se aplica à arena.

Se existe uma estrutura do robô que pode atingir um elemento do campo, o software pode verificar a posição antes de permitir determinada ação.

O palestrante cita inclusive mecanismos que podem ser automaticamente recolhidos antes de passar sob obstáculos.

Isso conversa diretamente com aquilo que observamos na Madtown, quando discutimos a colisão da Limelight com a trench.

A ideia da palestra é:

Não fazer o piloto gastar capacidade mental lembrando que precisa recolher determinado mecanismo.

Se for possível detectar e proteger automaticamente, o software deveria assumir essa responsabilidade.

17. IMU também pode ser utilizada como dispositivo de segurança

Foi apresentado um exemplo de 2022 no qual a equipe utilizava a **IMU para verificar o comportamento do robô durante a escalada.**

Caso o robô não tivesse engatado corretamente na próxima barra, o sistema poderia impedir a continuação automática da sequência.

Isso mostra outra aplicação interessante dos sensores:

Não utilizar sensores apenas para controlar posição.

Também utilizá-los para responder:

É seguro executar a próxima ação?

18. O software também deve impedir penalidades

Segurança não significa somente evitar danos físicos.

Pode significar também **impedir que o piloto viole regras do jogo.**

Se o jogo permite apenas determinada quantidade de peças e o robô já atingiu esse limite: simplesmente não permita outra coleta.

O piloto pode pressionar o botão, mas o software sabe que aquela solicitação não pode ser executada.

Assim:

Controle incorreto ≠ penalidade automática.

Ergonomia do controle

19. Os melhores botões são aqueles que não exigem remover os dedos dos analógicos

A prioridade apresentada foi aproximadamente:

Gatilhos / bumpers / botões traseiros → ABXY → D-Pad e botões menos acessíveis.

Os comandos mais importantes devem estar disponíveis sem retirar os polegares dos analógicos.

Isso é extremamente importante durante coleta e movimentação.

O piloto não deveria precisar **parar de dirigir para operar um mecanismo.**

Isso bate diretamente com algo que observamos na Citrus Circuits: os comandos utilizados durante coleta eram acessíveis sem comprometer o movimento principal do robô.

20. Evitar clicar nos analógicos

A recomendação da 604 é simplesmente:

não utilizar L3/R3 para ações importantes.

Ao pressionar o próprio analógico, o piloto pode involuntariamente alterar seu comando de movimento.

Na equipe, segundo o palestrante, esses botões são praticamente proibidos.

21. D-Pad para funções menos frequentes

Como utilizar o D-Pad normalmente exige retirar o polegar esquerdo do analógico de movimentação, ele deveria ficar reservado para funções como:

- override;
- desengasgar mecanismo;
- recuperação;
- funções raramente utilizadas.

A lógica é simples:

Se o piloto tirou o dedo do controle de translação, provavelmente deixou de avançar na tarefa principal do jogo.

22. Desgaste físico dos controles importa

Esse foi um detalhe pequeno, mas muito interessante.

Com o uso, o joystick pode deixar de retornar perfeitamente para zero.

Por exemplo:

Joystick solto → 6%

A solução comum seria aumentar a deadband.

Porém:

6% → 8% → 10%...

Quanto maior a deadband, pior fica a região de controle fino.

A equipe chegou à conclusão de que prefere **substituir controles desgastados durante a temporada** em vez de continuar aumentando a zona morta.

Também foi destacado que Hall Effect não necessariamente resolve esse problema, porque o desgaste citado pode estar no **mecanismo/mola de retorno**, não no sensor do joystick.

23. Resposta do joystick deve favorecer memória muscular

A equipe prefere uma resposta bastante previsível entre:

movimento do analógico → velocidade do robô.

A justificativa é memória muscular.

Se o mesmo deslocamento do joystick produz sempre aproximadamente o mesmo efeito, o piloto consegue internalizar melhor o comportamento.

Eles utilizam uma solução que combina resposta próxima da linear com tratamento adequado da deadband, evitando um salto brusco imediatamente após a zona morta.

24. Logar também os controles do piloto

Essa é uma aplicação excelente do AdvantageKit.

Não basta registrar:

- pose;
- corrente;
- velocidade;
- sensores;
- câmera;
- mecanismos.

Também devemos registrar:

joysticks + triggers + botões.

Porque depois da partida alguém pode dizer: “O robô virou muito devagar.”

Com o log podemos descobrir: **Problema do chassi?**

ou

Piloto estava comandando apenas 70%?

O palestrante conta justamente um caso em que o piloto reclamava da velocidade de rotação e, ao verificar os logs, descobriram que ele estava colocando o analógico aproximadamente a 45°, reduzindo o comando efetivo.

25. Treinar desde cedo com o layout definitivo

Outro aprendizado que considero muito importante para nossos times:

não utilizar controles provisórios por muito tempo.

Se durante os primeiros testes fazemos:

A → mecanismo 1

B → mecanismo 2

X → mecanismo 3

e depois mudamos tudo para:

RT → coletar,

o piloto já começou a construir uma memória muscular errada.

A recomendação é implementar **o layout definitivo o mais cedo possível**, mesmo que a automação por trás dele ainda não esteja completamente pronta.

26. Mudar cedo e travar cedo

O piloto deve participar do desenvolvimento do controle.

Durante os treinos:

- perguntar o que está estranho;
- verificar ergonomia;
- analisar logs;
- ajustar sensibilidade;
- ajustar comandos;
- melhorar feedback.

Mas essas mudanças devem acontecer **cedo**.

Depois que a memória muscular começa a ficar consolidada, grandes mudanças no meio da temporada podem gerar erros justamente sob pressão.

Conclusão

Dessa palestra, eu transformaria os conceitos em um padrão de desenvolvimento:

1. **Driver participa do projeto desde o CAD/concepção.**
2. Cada mecanismo deve responder: *como isso será controlado?*
3. Priorizar um único piloto para as ações principais.
4. Mapear **intenções**, não motores.
5. Collect, Score, Aim, Climb em vez de MotorUp, PivotDown, RollerOn.
6. Automatizar sequências repetitivas.
7. Utilizar rumble como feedback.
8. Utilizar LEDs para estados que também precisam ser visíveis externamente.
9. Evitar toggles sempre que possível.
10. Criar interlocks contra autocolisão.
11. Criar proteções contra colisões previsíveis com o campo.
12. Utilizar sensores também para validar segurança.
13. Impedir estados que causariam penalidades previsíveis.
14. Colocar ações frequentes em triggers/bumpers.
15. Evitar L3/R3.
16. Reservar D-Pad para overrides e ações raras.
17. Registrar entradas do controle no AdvantageKit.
18. Verificar desgaste físico dos controles.
19. Começar treino com o layout definitivo o quanto antes.
20. Depois de validado, **travar o layout para consolidar memória muscular.**

Principal aprendizado

Para mim, a frase que melhor resume a palestra seria:

O piloto deve controlar a intenção; o software deve controlar a execução.

O piloto não deveria pensar:

“abaixa intake → liga roller → espera sensor → recolhe intake → liga indexer → gira shooter → mira → dispara.”

Ele deveria pensar: **“Quero coletar.”**

e depois: **“Quero pontuar.”**

Todo o restante é responsabilidade do software, dos sensores e das proteções que projetamos.

Isso também explica algo que vimos claramente em equipes como Citrus, 254 e Under Control: **um controle de alto nível não necessariamente possui mais comandos; muitas vezes possui menos comandos, mas cada comando representa uma intenção muito maior do robô.**